

Sortowanie bąbelkowe

„[] opanowanie teorii jest zaledwie niezbędnym wstępem”

Metoda

Metoda

- Sortowanie bąbelkowe polega na porównywaniu sąsiadujących ze sobą elementów i zamianie miejscami tych, które są w niewłaściwej kolejności – po każdym przejściu maksymalny lub minimalny element (zależnie od warunku sortowania) jest przesuwany na koniec

Metoda

7	3	2	9	1
---	---	---	---	---



Porównujemy pary

3	7	2	9	1
---	---	---	---	---



Większy przesuwany jest na koniec

Metoda

3	7	2	9	1
---	---	---	---	---



Porównujemy pary

3	2	7	9	1
---	---	---	---	---



Większy przesuwany jest na koniec

Metoda

3	2	7	9	1
---	---	---	---	---



Porównujemy pary

3	2	7	9	1
---	---	---	---	---



Brak przesunięcia

Metoda

3	2	7	9	1
---	---	---	---	---



Porównujemy pary

3	2	7	1	9
---	---	---	---	---



Większy przesuwany jest na koniec

Metoda

3	2	7	1	9
---	---	---	---	---

- Ciąg nie jest posortowany, ale największa wartość znalazła się na końcu

Metoda

3	2	7	1	9
---	---	---	---	---

- Czynność porównania wyrazów w ciągu powtarzamy, ale już nie sprawdzamy dla ostatniej wartości w ciągu

Specyfikacja algorytmu

Specyfikacja

Dane:

- Liczba naturalna $n > 0$
- n -elementowa tablica jednowymiarowa zawierająca liczby rzeczywiste $T[0 \dots n-1]$

Wynik:

- Posortowana n -elementowa tablica jednowymiarowa zawierająca liczby rzeczywiste

Algorytm

```
dla i ← 1, 2, ..., n - 1 wykonuj
    dla j ← 0, 1, ..., n - i - 1 wykonuj
        jeśli A[j] > A[j+1] to
            pom ← A[j]
            A[j] ← A[j+1]
            A[j+1] ← pom
```

Implementacja

Implementacja

```
void bubbleSort(int array[], int n)
{
    for (int i = 1; i < n; ++i) {
        for (int j = 0; j < n - i; ++j) {
            if (array[j] > array[j + 1]) {
                int temp = array[j];
                array[j] = array[j + 1];
                array[j + 1] = temp;
            }
        }
    }
}
```

Procedura w stylu C

```
#include <iostream>
#include <vector>

void bubbleSort(std::vector<int>& array)
{
    int n = static_cast<int>(array.size());

    for (int i = 1; i < n; ++i)
    {
        for (int j = 0; j < n - i; ++j)
        {
            if (array[j] > array[j + 1])
            {
                int pom = array[j];
                array[j] = array[j + 1];
                array[j + 1] = pom;
            }
        }
    }
}
```


Implementacja

```
int main()
{
    std::vector<int> vecArray{64, 34, 25, 12, 22, 11, 90, 5, 78, 45};
    bubbleSort(vecArray);

    std::cout << "Sorted array: \n";
    for (int i = 0; i < vecArray.size(); ++i) {
        std::cout << vecArray[i] << " ";
    }

    std::cout << std::endl;
    return 0;
}
```

Złożoność algorytmu

Złożoność algorytmu

- Operacją dominującą w algorytmie bąbelkowym jest porównanie
- Pętla zewnętrzna powtarzana jest $n-1$ razy
- W pętli wewnętrznej wykonywanych jest $n-1$ porównań, w kolejnym kroku $n-2$, a w ostatnim tylko jedno porównanie

Złożoność algorytmu

$$(n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2}$$



$$\text{od } 0 \text{ do } \frac{n(n-1)}{2}$$

Co daje złożoność $O(n^2)$

Złożoność algorytmu

- Wadą algorytmu jest to, że liczba porównań nie zależy od wartości wyrazów ciągu liczbowego - algorytm można zmodyfikować w taki sposób, aby zmniejszyć liczbę porównań: jedną z takich możliwości jest zakończenie algorytmu w przypadku stwierdzenia, że ciąg jest posortowany

www.szymkowiak.tychy.pl

KONIEC

Grzegorz Szymkowiak